

Real-world (compare your loop to the library's) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `agent_executor_comparison_practice.py`: now that you've built the primitive by hand, seeing the library's version side by side tells you exactly what `create_agent` is doing for you, and what it's hiding. **If not:** the first time you'd meet `create_agent`'s behavior would be in real Project 2 code, with no smaller comparison to check your assumptions against.

All examples below reuse `get_weather` and `celsius_to_fahrenheit` from `build_react_loop`, and assume `run_agent()` from that exercise's Solution is already in scope.

Basic Version

```
# agent_executor_comparison_practice.py
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

@tool
def get_weather_tool(city: str) -> str:
    """Get the current weather for a city, in Celsius."""
    return get_weather(city)

llm = ChatOpenAI(model="gpt-4o-mini")
agent = create_agent(llm, tools=[get_weather_tool])

result = agent.invoke({"messages": [("user", "What's the weather in Paris?")]}))
print(result["messages"][-1].content)
```

Expected output (final line; use `.stream(inputs, stream_mode="updates")` instead of `.invoke()` if you want to watch each step happen live):

It's 18°C and cloudy in Paris right now.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — 3 test questions, both loops, diffed

```
# agent_executor_comparison_practice.py
test_questions = [
    "What's the weather in Paris?",
    "What's the weather in Paris, in Fahrenheit?",
    "What's the capital of France?",
]

for question in test_questions:
    mine = run_agent(question)
    lib = agent.invoke(
        {"messages": [("user", question)]}, {"recursion_limit": 11}
    )

    tool_call_count = 0
    for m in lib["messages"]:
        if getattr(m, "name", None):
            tool_call_count = tool_call_count + 1

    print("QUESTION:", question)
    print("  mine:", mine.final_answer, "|", len(mine.steps), "tool call(s)")
    lib_answer = lib["messages"][-1].content
    print("  lib: ", lib_answer, "|", tool_call_count, "tool call(s)")
    print()
```

Expected output (shape; exact wording varies run to run):

```
QUESTION: What's the weather in Paris?
mine: It's 18°C and cloudy in Paris right now. | 1 tool call(s)
lib:  It's 18°C and cloudy in Paris right now. | 1 tool call(s)

QUESTION: What's the weather in Paris, in Fahrenheit?
mine: It's 64.4°F in Paris. | 2 tool call(s)
lib:  It's 64.4°F in Paris right now. | 2 tool call(s)

QUESTION: What's the capital of France?
```

```
mine: The capital of France is Paris. | 0 tool call(s)
lib:  The capital of France is Paris. | 0 tool call(s)
```

Approach 2 — counting the library's steps directly, not just eyeballing the stream

```
# agent_executor_comparison_practice.py
result = agent.invoke(
    {"messages": [("user", "What's the weather in Paris, in Fahrenheit?")]},
    {"recursion_limit": 11},
)
print(result["messages"][-1].content)

tool_messages = []
for m in result["messages"]:
    if getattr(m, "name", None):
        tool_messages.append(m)
print("lib step count:", len(tool_messages))
```

`result["messages"]` holds the whole conversation, including every tool call and its result — filtering for messages that have a `name` set (a tool's own return message) gives you the library's own equivalent of your `steps` list, so you can compare step *counts* precisely instead of eyeballing a printed stream. **Expected output:**

```
It's 64.4°F in Paris right now.
lib step count: 2
```

Difference from Basic: Basic runs one question through the library loop alone. Approach 1 runs the *same* 3 questions through both loops and prints them side by side, so you can actually compare, not just observe each one in isolation. Approach 2 replaces "reading a printed stream" with reading `result["messages"]` directly — the more reliable way to compare step counts, since streamed output is meant for humans to read, not for code to parse.

Two real differences worth knowing, if you push the comparison further: a `create_agent`-built agent catches a tool's exception on its own and feeds the error text back to the model — your own hand-built `run_tool()` does the same *shape* of thing, but feeds back the exact message you chose (`f"Error: {e}"`), which may not be word-for-word what the library sends. And once `recursion_limit` is hit, `create_agent` raises `GraphRecursionError` (it's built on `LangGraph` underneath) — the same "fail loudly instead of quietly returning a stopped-early message" behavior your own `MaxIterationsExceeded` already gives you, so both loops agree on this point.

Which one should you actually use? For Project 2 and most real work: `create_agent` (or hand-built `LangGraph`, covered later) — it's tested, handles edge cases you'd otherwise reinvent, and having built `run_agent()` yourself means you now know what it's doing well enough to trust it, and

precisely enough to know where to look when it does something unexpected. Keep your own hand-built loop only as a reference, or for a case needing a specific behavior — like a very particular error message on a tool failure — that you want full control over instead of accepting whatever the library sends by default.